M3: Making the book fast, and proving it is the same book

itchbook walkthrough, milestone 3. How the reference book's 336 ns per message became 92, why each change helped or didn't, including the optimisation that was switched off at the end, what the comparison with CppTrader does and doesn't show, and how the fast engine was held to byte-identical output.

What M3 had to show

M2 produced a book that is right: no orphans, an empty book at the close, and checksums identical to an independent Python implementation. It is also slow, because it was written to be easy to check rather than fast. M3 had three requirements. First, a second engine, `FastBook`, whose output is identical to the reference book's, every counter, every level, every queue position and every checksum. Second, a throughput number measured against an established open-source engine, CppTrader, on the same file, the same machine and the same core. Third, a latency distribution per book update (p50, p99, p99.9) measured honestly, with the timer's own cost reported next to it.

The order matters. A fast book that is slightly wrong is worth nothing to M4, which needs the true queue position of every order. So the fast engine was built against the reference book as an oracle from the first line, and every speed change was accepted only if the oracle still agreed.

Where the time goes in a book update

To make a book fast you have to know what it waits for. A NASDAQ day is 423 million messages, and the book's working set is large: up to 1.9 million live orders spread over thousands of symbols, plus an index from order reference number to order. That is far bigger than the CPU's caches. Arithmetic on a message takes a few nanoseconds. A load that misses every cache and goes to DRAM costs 80 to 100 ns. So the cost of an update is almost entirely the number of cache misses on its critical path, and how many of them must happen one after another.

Consider an execution. The message carries only a reference number. To apply it the book must (1) look the reference up in the index, which gives the order's location; (2) load the order, which gives its price level; (3) load the level and update its share total; and, if the order is removed, (4) load its neighbours in the queue to unlink it. Each load depends on the one before, because you don't know where to look next until the previous load returns. Four dependent misses at 90 ns each is 360 ns, which is roughly what the reference book costs. Everything in this milestone is an attack on

that chain: make each step touch fewer cache lines, remove steps, and start the loads early enough that they have finished by the time the message is processed. The table below is the whole story in numbers, on the cloud machine described later, one pinned core, nanoseconds per message over the full day. The sections after it explain each row.

Version	ns per message
ReferenceBook (M2): ordered maps, linked lists	336, 356 (two runs)
FastBook, hash index, no prefetch	164–173
FastBook, hash index, two-stage prefetch	154–158
paged index, two-stage prefetch	120–124
+ level pool, two-stage prefetch	99–109
+ three-stage prefetch (reverted)	109–114
level pool, prefetch off (final)	91–97

The fast engine's layout

`FastBook` keeps the same five operations as the reference book (add, execute, cancel, delete, replace) and changes only how the data is stored.

Orders live in one pooled array. Each order is a 32-byte slot, so two fit in one 64-byte cache line: reference, price, shares, symbol, side, the index of its price level, and the indices of the previous and next orders in its queue. Orders are addressed by 32-bit index rather than pointer. A free list recycles slots, so once the pool has grown to the day's peak the hot path never calls the allocator. The reference book allocated a map node and a list node on every add.

Each price level's queue is an intrusive doubly linked list. The previous and next links live inside the order slots, so walking or unlinking a queue touches only order slots, and removing an order from the middle of a queue is constant time, which time priority requires.

Price levels live in a second pool, and every order stores its level's index. This is the change that removed a step from the chain. An execution, cancel or delete goes from the order straight to its level, with no search by price.

Each side of each symbol's book is a vector of (price, level index) pairs sorted with the best price at the back. Almost all activity is at or near the top of the book, so putting the best price at the end of the vector means a new level near the top is inserted by moving a handful of elements, not the whole side. The vector is searched only when a level is created or disappears. The search scans the last eight entries first and falls back to binary search for orders deep in the book.

Symbols are indexed by stock locate, the dense small integer NASDAQ assigns each symbol for the day, so finding a symbol's book is an array lookup.

The order index: from hash table to paged direct table

The index maps a 64-bit order reference to a slot. The first version was an open-addressing hash table with linear probing and backward-shift deletion: flat, no pointers, sized to twice the live orders. It works, but every lookup is a random access into a 64 MB table. A random access into 64 MB misses the cache almost every time, so step (1) of every chain cost a DRAM trip.

The data suggested something better. NASDAQ assigns order references sequentially through the day: 01302020 uses references up to 494 million, densely. A direct table indexed by the reference itself needs no hashing and no probing, and, more importantly, it has locality. Consecutive adds write to neighbouring entries, and most executions and cancels refer to orders added recently, whose entries are still in cache. The problem is size: a flat table of 494 million 4-byte entries is 2 GB, and a hostile or corrupted reference of 2^60 would ask for an impossible allocation.

`PagedRefIndex` solves both. The table is split into pages of 65,536 entries (256 KB), allocated the first time a reference lands in them. A page is released when its last live order dies, unless it is the newest page, so a page at the frontier isn't freed and reallocated over and over. Memory therefore follows the span of references that still have live orders, not the largest reference ever seen. References at or above 2^36, which never occur in real files but do in fuzzed ones, go to a small hash table fallback, so a single huge reference can't make the page table explode. The fuzzer exercises exactly that path. The guard has a limit worth knowing: a stream built to keep one live order in each of a million different pages below 2^36 would still allocate a page for each. That is acceptable for a replay engine reading exchange files, and a hard cap on live pages, spilling to the fallback, would close it.

On the cloud machine the switch took the engine from about 156 to about 122 ns per message, the largest single gain after leaving the reference book. The index's peak memory on 01302020 was 1,964 MB, and that number deserves an honest reading. It is about 7,500 pages, nearly all of the 7,545 that the day's 494 million references span, so on this day paging saved almost nothing over a flat 2 GB table. Long-lived orders are scattered through the whole day, and one live order is enough to keep its page. The speed comes from the direct mapping, not the paging. What paging buys is safety: memory grows with the references actually in use rather than with whatever number a message claims, and pages are released on a day whose old orders do die out. Two gigabytes still fits comfortably on an 8 GB laptop.

Software prefetching

Even with fewer misses, each update still waits for some. The way around a dependent chain is to start it early. The parser already holds the next few hundred kilobytes of the file in its buffer, so it can look ahead. The stream reader keeps two cursors running ahead of the message being decoded. When a frame is 16 messages ahead (stage 0), the book is asked to prefetch what can be located from the message alone: the index entry for its reference (and for the new reference of a replace), and, for an add, the symbol's book header. When the same frame is 8 messages ahead (stage 1), the index entry has arrived, so the book can follow it and prefetch the order slot; for an add it prefetches the best level of the side the order joins.

Prefetch instructions are hints. They never change what the program computes, only how long loads take, which is why the property tests and the fuzzer run with prefetching on and the output is still compared with the reference. On the hash-index engine prefetching was worth 8 to 19 ns per message across three interleaved rounds, about 10 at the median (154 to 158 ns with it, 164 to 173 without). The reason is memory-level parallelism: a core can have many misses outstanding at once, and prefetching turns a series of misses, one per message, into overlapping ones.

It isn't free, though. The lookahead cursors read every frame's length and type two extra times, and stage 1 does a real (demand) load of the index entry so it can find the slot to prefetch. If that entry hasn't arrived yet, the parsing thread stalls on it, which is exactly the wait prefetching was meant to remove. Whether prefetching pays depends on how many misses there are left to hide. That turned out to matter later.

The level pool

A `perf` profile of the engine before this change (taken on the hash-index build) showed two functions near the top: removing an order (21% of samples) and finding its price level (18.5%). The profile itself wasn't saved; the numbers are from the session notes. In that version, levels were stored inline in each side's sorted vector, so an execution had to search the vector by price to find its level. Inserting a level shifted the others, so an order couldn't hold a stable reference to its level.

Moving levels into their own pool, with the sorted vector holding only (price, index) pairs, fixed both. Orders store the index of their level, which never moves while the level exists, so the price search disappears from every execution, cancel and delete. The sorted vector holds 8-byte pairs instead of full levels, so it is denser and cheaper to shift. This took the engine from about 122 to 99 to 109 ns per message, over two separate interleaved runs.

A change that didn't work, and was reverted

The obvious next step was a third prefetch stage: index entry, then order slot, then the order's level and its queue neighbours, which are the remaining dependent loads of a delete. It measured slower: 109 to 114 ns against 99 to 109 ns for two stages, in back-to-back runs on the same core (each run interleaved with CppTrader and the hash index, not with each other). The likely reasons are that the extra stage re-reads and re-parses every frame a third time, and that its lookup of the order slot stalls whenever stage 1's prefetch hasn't completed, which is exactly the latency it was supposed to hide. It was reverted, and the commit message records the numbers.

Turning prefetching off

The final head-to-head run against CppTrader includes a control: the same engine with no prefetch instructions issued. On the final engine the control was faster, 94 against 101 ns. Rather than guess, `itch-bench` gained a switch for each stage and for the lookahead pass itself, and `scripts/bench_prefetch_ab.sh` measured them in interleaved rounds: both stages, stage 0 only, stage 1 only, lookahead with no prefetches, and no lookahead at all. A five-round confirmation followed:

Configuration	ns per message, 5 interleaved rounds	median
two-stage prefetch	100.6, 105.4, 108.4, 99.8, 104.5	104.5
no lookahead, no prefetch	94.0, 96.5, 90.7, 91.8, 91.2	91.8

Plain decode-and-apply won in every round, by 7 to 18 ns. The explanation is the same one that made prefetching useful in the first place, read the other way round. The paged index made the index lookup mostly a cache hit, because recent references sit in recently touched pages. The level pool removed the price search. With few misses left to hide, what remains is the cost of the lookahead itself: two extra passes over the frame headers and a stall in stage 1 whenever its demand load misses. So prefetching is now off by default and `--prefetch` turns it back on. The code stays, tested, because it is still the right choice for the hash index and for any future change that brings misses back. This is the lesson worth giving in an interview: an optimisation is a claim about a particular bottleneck, and when other changes remove that bottleneck the claim has to be measured again.

Proving it is still the same book

Speed changes are where subtle bugs come from, so `FastBook` is held to the reference book at every level of testing.

Differential property tests. RapidCheck generates random event streams over a tiny universe (a few symbols, a few prices, many collisions, many references to orders that don't exist), feeds the same stream to both engines, and compares every

counter, every level, every queue rank and the whole-book checksum after every event. The order index has its own property tests against `std::unordered_map`, including a test that releasing emptied pages keeps every live order findable.

The fuzzer drives the fast book, prefetching switched on, with arbitrary bytes under AddressSanitizer and UndefinedBehaviorSanitizer, including the huge references that take the hash fallback.

The full-day gate. `itch-book --engine fast` on 01302020 produces output byte-identical to the M2 reference book: the same 2,298 lines of counters, checksums at 35 times of day and top-of-book snapshots, SHA-256 `baa1b463…`. This was rerun on commit 886e798, after the last change to the book or the parser. The only later changes are to the benchmark tool's prefetch switches, the scripts and the documentation, none of which `itch-book` uses.

The comparison with CppTrader

CppTrader is an open-source C++ trading platform whose README publishes ITCH benchmark numbers, which makes it the natural reference. Its numbers were measured on a different machine years ago, so quoting them next to ours would mean nothing. The script `scripts/bench_cpptrader.sh` builds CppTrader at a pinned commit and runs both engines on the same uncompressed file, on the same machine, pinned to the same core, in interleaved rounds so that thermal throttling or noisy neighbours hit both alike. Both read the file on the measured thread, as CppTrader's own benchmark does, which is what `itch-bench --inline-input` is for. Both numbers are total processing time divided by total messages. CppTrader also prints a "market update latency", which divides by its count of market updates (916,566,126 on this day, more than twice the 423,285,709 messages, because one message can produce several updates) and is not comparable.

The final runs were on an AWS m7i.2xlarge (Intel Xeon Platinum 8488C, 8 vCPUs, 30 GB) under Ubuntu 24.04 and GCC 13, pinned to CPU 2, on 01302020 (423,285,709 messages, 12.95 GB uncompressed). The ranges are over interleaved rounds:

Engine	What it maintains	ns per message
CppTrader `itch_handler`	parsing only, no book	17
CppTrader `market_manager`	L3 book, time priority	309–321
CppTrader `market_manager_optimized`	aggregated levels only	75–82 (median 77)
itchbook `ReferenceBook` (M2)	L3 book, time priority	336, 356 (two runs)
itchbook `FastBook` (final)	L3 book, time priority	91–97 (median 92)

The CppTrader rows and the final `FastBook` row come from two runs about half an hour apart on the same machine. In the run where they were interleaved directly,

CppTrader's L3 book took 309 to 321 ns and `FastBook` with only the lookahead pass (no prefetches, slightly slower than the final default) took 94 to 98, a ratio of 3.3. A cloud machine shares its physical host with other customers, so round-to-round noise of about 5 ns is normal. That is why every comparison here is interleaved and quoted as a range, and why a difference smaller than the range isn't claimed.

CppTrader has two books, and the difference between them matters for reading this table.

`market_manager` is the like-for-like comparison. It keeps every order in time priority at its level, which is what M4 needs and what `FastBook` does. On the same file and core, `FastBook` is about 3.3 times faster.

`market_manager_optimized` is faster than `FastBook`, and does less. It keeps aggregated levels only, with no queue of orders, so it can't say where an order stands in line, which is the quantity M4 measures. Its order table is a flat array indexed directly by reference number, sized in the source for 300 million orders, with no bounds check. On 01302020 references reach 494 million, so unmodified it would write past the end of its array; the script enlarges that one constant to 500 million and changes nothing else. At that size the array alone is about 16 GB, which is why it can run only on the cloud box and not on an 8 GB laptop. `FastBook`'s whole index peaks at 2 GB. So the honest reading is: on an L3 book with time priority, itchbook is about 3.3 times faster than CppTrader's equivalent; CppTrader's aggregated book is about 15 ns per message faster, by skipping the queue and spending eight times the memory.

On the MacBook Air (M2, 8 GB, no core pinning on macOS), where the optimised book can't run at all:

Engine	ns per message
CppTrader `market_manager` (L3, time priority)	224–254
itchbook `FastBook`, final default (2 rounds)	89–97
itchbook `FastBook`, prefetch on (3 rounds)	94–98

That is about 2.5 times faster. The laptop was in everyday use during these runs, so they are noisier than the cloud ones. In one round everything ran about twice as slow (CppTrader 511 ns, itchbook 181 ns) while another program was busy. That round is left out of the ranges and kept in the results file. The ratio between the engines is the number to trust here, not the absolute values.

Latency per update

Throughput is the average. A trading system also cares about the tail: the slowest one in a hundred or a thousand updates. `itch-bench --latency` reads a timer before and after every book update and records the difference in a log-linear histogram in

the style of HdrHistogram. Values below 256 ticks are counted exactly, above that there are 128 sub-buckets per power of two, so every percentile is within 0.8% of the true value. Recording never allocates.

On x86 the timer is the time-stamp counter (`rdtsc`), converted to nanoseconds with a rate calibrated against the system clock over the whole run, so no CPU frequency is assumed. The cost of reading it twice is measured and printed (about 13 ns on the Xeon), because that cost is in every sample. Timing every update also slows the run overall, so throughput is always reported from a separate run without `--latency`.

Per-update latency on the Xeon for every book update of the day, with the final default and, for comparison, with prefetching on:

Percentile	prefetch off (default)	prefetch on
p50	54 ns	47 ns
p90	242 ns	212 ns
p99	517 ns	473 ns
p99.9	743 ns	697 ns
p99.99	11.1 µs	10.8 µs

Each column is from its own run: prefetch off on commit 3c43e56 (timer cost 12.5 ns), prefetch on on 886e798 (13.3 ns). Every sample includes that timer cost.

The two columns show something worth understanding. With prefetching on, each individual update is faster at every percentile, yet the whole day takes longer. Both are true because the timer brackets only the book update. The prefetch work happens in the parser's lookahead pass, outside the timed region. It moves loads earlier, so the update itself finds more data in cache, but the lookahead costs more in total than it saves. Which one you want depends on the system. A replay or research engine like this one cares about throughput, so prefetching is off. A live engine that must react to one message as fast as possible might take the trade the other way, because its lookahead could run while it would otherwise be idle, waiting for the next packet.

The median is low because most updates find their data in cache: recent orders, recent index pages, levels at the top of the book. The tail is the updates where they don't: a miss chain all the way to DRAM, or inserting a new level into a long side. The p99.99 and maximum are dominated by events outside the book entirely, such as page faults when a new index page is first touched and the operating system's interrupts, which is why the run is pinned to a core away from CPU 0.

On Apple silicon the equivalent counter ticks at 24 MHz, one tick per 41.7 ns, so per-update latencies there are quantised, and the benchmark prints the resolution on its `timer` line. The Xeon numbers are the ones to quote.

What M3 leaves for later

The remaining gap to CppTrader's aggregated book is mostly the queue itself: unlinking an order touches its neighbours, which an aggregated book never has to do. Closing it further would mean changing the layout of the queue (for example, storing each level's orders in a small array instead of a linked list), which makes deletes from the middle of a queue more expensive. M4 needs queue positions, not another 15 ns, so M3 stops here.

Questions to expect

Your engine is slower than CppTrader's optimised book. Why claim a win? Because they solve different problems. The optimised book keeps only the total at each price; it can't answer "how many shares are ahead of this order", which is the whole point of an L3 book. Against CppTrader's L3 book, measured on the same file and core, itchbook is about 3.3 times faster, and it uses an eighth of the memory of the optimised book while doing more.

How do you know the fast book is correct? It isn't trusted on its own. Random differential tests compare it with the reference book after every event, the fuzzer runs it under sanitizers, and on a full trading day its output is byte-identical to the reference book's, which in turn matched an independent Python implementation in M2.

Why a paged table instead of a hash table? Because order references are assigned sequentially, so the reference itself is a near-perfect index with locality. A hash function deliberately destroys that locality. Pages keep memory tied to the references actually in use, and the hash fallback stops a single absurd reference from exhausting memory. Be ready to say that on 01302020 the paging saved almost no memory: the speed comes from the direct mapping.

Why is prefetching off if it helped? It helped while the index lookup was a random access into a 64 MB hash table. After the paged index and the level pool removed most of those misses, the lookahead pass cost more than it hid: plain decode-and-apply won all five interleaved rounds by 7 to 18 ns. The same reasoning killed the three-stage version earlier. A stage that depends on an earlier prefetch stalls if that prefetch hasn't returned.

How do you measure a 60 ns event without the measurement dominating it? With a cycle counter that costs about 13 ns per pair of reads, reported alongside the results, and by never quoting throughput from a timed run.